Manual técnico — ExtreamFS

Universidad de San Carlos de Guatemala
Manejo e Implementación de Archivos — Proyecto 2 — 2S2026
Estudiante: Rodrigo Mérida — Carné: 202000900 — Sección: D
Auxiliar: Natalia Calderón

1. Propósito y alcance

ExtreamFS administra discos binarios `.mia` y simula particiones y sistemas de archivos EXT2/EXT3. El backend C++17 trabaja directamente sobre el archivo binario; no mantiene un árbol de archivos en memoria. Solamente los montajes conocidos y una sesión activa son volátiles. El frontend React permite ejecutar scripts, administrar la sesión, visualizar discos, explorar el sistema de archivos y consultar el journal.

El archivo de disco se crea con su tamaño definitivo y las operaciones respetan los límites reservados. El proyecto se ejecuta en GNU/Linux y está preparado para alojar el backend en Amazon EC2 y el frontend estático en Amazon S3.

2. Arquitectura

```
Navegador / React + Vite
| HTTP JSON
v
API REST C++ :8080
| parser y despachador
v
Motor ExtreamFS
|-- MBR / EBR
|-- EXT2 o EXT3 + Journal
|-- permisos UGO y usuarios
`-- Graphviz
|
v
discos .mia y reportes
```

Flujo de una operación:

1. React envía comandos o una consulta de vista a la API.
2. La API valida la ruta, extrae el JSON y llama al motor compartido.
3. El parser conserva comentarios, admite parámetros entre comillas y normaliza el nombre del comando.
4. El motor valida montaje, sesión, permisos, capacidad y nombres antes de escribir.
5. Los cambios se realizan directamente en el disco `.mia`; en EXT3 se agrega una entrada al journal cuando corresponde.
6. La respuesta JSON actualiza la terminal o la vista gráfica.

3. Organización del código

Componente	Ubicación	Responsabilidad
API REST	`backend/src/main.cpp`	Servidor, CORS, rutas y JSON
Motor	`backend/src/extreamfs.cpp`	Parser, comandos, disco, permisos y reportes
Contrato	`backend/include/extreamfs.hpp`	Interfaz pública, montajes y sesión
Estructuras	`backend/include/structures.hpp`	Representación binaria empacada
Pruebas	`backend/tests/ext3_tests.cpp`	Regresión de EXT2, EXT3 y comandos P2
Interfaz	`frontend/src/main.jsx`	Terminal y vistas gráficas
Estilos	`frontend/src/style.css`	Diseño adaptable y accesibilidad visual
Confirmación	`frontend/src/confirmation.css`	Diálogo de borrado de partición

4. Estructuras binarias

Estructura	Uso
`MBR`	Tamaño, fecha, firma, ajuste y cuatro descriptores
`Partition`	Estado, tipo, ajuste, inicio, tamaño, nombre, correlativo e ID
`EBR`	Lista enlazada de particiones lógicas
`SuperBlock`	Totales, libres, tamaños, fechas y posiciones de las áreas
`Inode`	UID, GID, tamaño, tiempos, tipo, permisos y 15 apuntadores
`FolderBlock`	Cuatro pares nombre/inodo
`FileBlock`	Hasta 64 bytes de contenido
`PointerBlock`	Dieciséis apuntadores
`JournalInformation`	Operación, ruta, contenido y fecha
`Journaling`	Arreglo fijo y contador de 50 operaciones recientes

`i_block[0..11]` son directos, `i_block[12]` indirecto simple, `i_block[13]` doble y `i_block[14]` triple. La lectura, escritura, liberación y los reportes recorren estos niveles.

5. Distribución EXT2 y EXT3

EXT2:

```
SuperBlock | bitmap inodos | bitmap bloques | tabla inodos | bloques
```

EXT3:

```
SuperBlock | journal (50) | bitmap inodos | bitmap bloques | tabla inodos | bloques
```

El número de bloques es tres veces el de inodos. `mkfs` calcula la capacidad con los tamaños reales de los structs, inicializa bitmaps y crea `/`, `/users.txt`, el grupo `root` y el usuario `root` con contraseña inicial `123`.

El journal es circular: al llenarse conserva las 50 operaciones más recientes. Los parámetros `-pass` se redactan antes de guardar o mostrar el comando.

6. Comandos implementados

Área	Comandos
Discos	`mkdisk`, `rmdisk`
Particiones	`fdisk` P/E/L, `-add`, `-delete=fast/full`, `mount`, `unmount`, `mounted`
Formato/recuperación	`mkfs -fs=2fs/3fs`, `journaling`, `loss`
Sesión	`login`, `logout`
Usuarios	`mkgrp`, `rmgrp`, `mkusr`, `rmusr`, `chgrp`
Archivos	`cat`, `mkdir`, `mkfile`, `remove`, `rename`, `copy`, `move`, `find`
Seguridad	`chown`, `chmod`
Reportes	`rep`

Reglas importantes:

- `fdisk -delete` exige `-confirm`; `full` también pone en cero la región.
- No se elimina un disco mientras tenga particiones montadas.
- `unmount` elimina el montaje y cierra la sesión si la utilizaba.
- `loss` funciona únicamente en EXT3; conserva superbloque y journal y limpia bitmaps, inodos y bloques. La sesión permanece activa para permitir el `logout` gráfico solicitado por las pruebas oficiales.
- `remove`, `rename` y `move` protegen `/` y `/users.txt`.
- No se sobrescriben destinos existentes ni se permite mover/copiar una carpeta dentro de sí misma.
- Los nombres de entrada que exceden los 12 bytes de `b_name[12]` se rechazan; los nombres de exactamente 12 caracteres se conservan completos.
- `find` admite `*` para uno o más caracteres y `?` para un carácter.

7. Permisos y consistencia

Los permisos se interpretan como UGO octal. Para cada operación se selecciona propietario, grupo u otros; `root` mantiene acceso total. `cat` y el explorador comprueban lectura. Las modificaciones validan escritura en el directorio padre. `chown` requiere `root` o ser propietario; `chmod` requiere `root` o ser propietario y acepta `-r`.

Las operaciones destructivas o recursivas prevalidan el árbol cuando es posible. La creación revierte asignaciones si no puede enlazar el nuevo inodo, reduciendo modificaciones parciales ante falta de espacio.

8. Reportes

`rep` admite `mbr`, `disk`, `inode`, `block`, `bm_inode`, `bm_block`, `tree`, `sb`, `file` y `ls`. Los reportes gráficos se construyen como DOT y Graphviz los convierte al formato indicado por la extensión. `file` y los bitmaps pueden producir texto. `disk` detalla EBR, particiones lógicas y espacios libres internos de la extendida.

9. API REST

Método y ruta	Entrada	Resultado
`GET /health`	—	Disponibilidad de la API
`GET /api/state`	—	Discos conocidos, particiones, montajes y sesión
`POST /api/execute`	`{"commands":"..."}`	Salida de uno o varios comandos
`POST /api/explorer`	`{"path":"/"}`	Directorio o archivo de la sesión activa
`POST /api/journal`	`{"id":"001A"}`	Entradas EXT3 del montaje

El servidor escucha en `0.0.0.0:8080`; otro puerto puede pasarse como primer argumento. Incluye encabezados CORS para el frontend en otro origen.

10. Interfaz

- Terminal: editor, carga `.smia`, ejecución y salida.
- Discos: estado de discos, particiones primarias/lógicas y montajes; el borrado solicita confirmación.
- Explorador: navegación de solo lectura y vista previa de archivos.
- Journal: tabla de operación, ruta, contenido y fecha.
- Sesión: formulario de ingreso, usuario activo y cierre de sesión.

`VITE_API_BASE_URL` define el backend en compilación. Si no existe, se usa `http://localhost:8080`.

11. Compilación y pruebas

```
cmake -S backend -B backend/build
cmake --build backend/build --clean-first -j2
ctest --test-dir backend/build --output-on-failure
cd frontend
npm ci
npm run build
```

La suite verifica distribución EXT2/EXT3, journal, redacción de contraseñas, pérdida, creación/redimensionamiento/eliminación de particiones, montaje lógico, permisos, operaciones de archivos y respuestas JSON. Una segunda prueba reproduce las cuatro partes oficiales publicadas el 13 de octubre de 2026.

12. Despliegue

La guía `deploy/AWS.md` deja el procedimiento completo. La topología mínima de evaluación es:

```
S3 website (frontend dist/) --HTTP/JSON--> EC2 :8080 (backend C++)
```

El grupo de seguridad debe limitar SSH a la IP del estudiante. El puerto de la API puede habilitarse temporalmente para evaluación. Para producción se recomienda un proxy HTTPS y restringir CORS; el endpoint de sitio web de S3 no ofrece HTTPS directamente.

13. Limitaciones conocidas

- El servidor atiende una solicitud a la vez y mantiene una sesión global; es apropiado para la demostración académica, no para múltiples usuarios simultáneos.
- `loss` simula la pérdida solicitada; el comando de recuperación no forma parte del alcance indicado.
- Montajes y sesión se pierden al reiniciar el backend; los discos `.mia` persisten.

14. Mantenimiento

- Ejecutar CTest y la compilación del frontend antes de cada entrega.
- Conservar el empaquetado y tamaño de las estructuras binarias.
- No registrar contraseñas, llaves ni credenciales.
- Eliminar recursos de AWS después de la calificación para evitar cargos.